

Bölüm 2 — Büyük Dillerin Evrimi (Chapter 2)

Programlama Dilleri

Büyük Dillerin Evrimi

Tüm Sorular — İngilizce & Türkçe + Cevaplar / Yönlendirmeler

Kaynak Kitap:

Concepts of Programming Languages

Robert W. Sebesta — 12. Baskı

63

Gözden Geçirme
Sorusu

24

Problem
Seti Sorusu

3

Programlama
Alıştırması

90

Toplam
Soru

- Gözden Geçirme Soruları (Review Questions)
- Problem Seti (Problem Set)
- Programlama Alıştırmaları (Programming Exercises)

BÖLÜM 2 — GÖZDEN GEÇİRME SORULARI (Review Questions) • 63 Soru**Soru 1**

EN: In what year was Plankalkül designed? In what year was that design published?

TR: Plankalkül hangi yılda tasarlandı? Bu tasarım hangi yılda yayımlandı?

● Cevap / Yönlendirme

Plankalkül, Konrad Zuse tarafından 1942–1945 yılları arasında tasarlanmıştır. Ancak tasarım, savaş koşulları ve Almanya'nın yenilgisi nedeniyle yalnızca 1972 yılında yayımlanabilmiştir. Bu gecikme, dilin öncü niteliğinin zamanında tanınmasını engellemiştir.

Soru 2

EN: What two common data structures were included in Plankalkül?

TR: Plankalkül'de hangi iki yaygın veri yapısı yer alıyordu?

● Cevap / Yönlendirme

Plankalkül, dizileri (arrays) ve kayıtları (records) destekleyen ilk dil olma özelliğini taşır. Bu iki veri yapısı, ileride COBOL ve Pascal gibi dillerde de temel bileşenler hâline gelmiştir.

Soru 3

EN: How were the pseudocodes of the early 1950s implemented?

TR: 1950'lerin başındaki sözde kodlar (pseudocodes) nasıl uygulandı?

● Cevap / Yönlendirme

1950'lerin başındaki sözde kodlar (pseudocodes), saf yorumlayıcı (pure interpreter) aracılığıyla uygulandı. Programlar doğrudan makine koduna derlenmez; bunun yerine her deyim çalışma zamanında yorumlanırdı. Bu yöntem yavaştı, ancak o dönem için kabul edilebilirdi.

Soru 4

EN: Speedcoding was invented to overcome two significant shortcomings of the computer hardware of the early 1950s. What were they?

TR: Speedcoding, 1950'lerin başındaki bilgisayar donanımının iki önemli eksikliğini gidermek için icat edildi. Bu eksiklikler nelerdi?

● Cevap / Yönlendirme

Speedcoding'in gidermeye çalıştığı iki donanım eksikliği şunlardır: (1) Kayan noktalı aritmetik (floating-point arithmetic) desteğinin olmaması — IBM 701'de doğrudan kayan noktalı işlem yapılamıyordu; (2) Belirli adresleme modlarının (index register ve indirect addressing) eksikliği — bu durum dizi ve döngü işlemlerini son derece zorlaştırıyordu.

Soru 5

EN: *Why was the slowness of interpretation of programs acceptable in the early 1950s?*

TR: 1950'lerin başında program yorumlamasının yavaşlığı neden kabul edilebilir düzeydeydi?

● Cevap / Yönlendirme

1950'lerin başında programcı zamanının bilgisayar zamanından çok daha değerli olduğu kabul ediliyordu. Programcıların makine kodunda manuel olarak program yazma süreci son derece meşakkatli ve zaman alıcıydı. Yorumlayıcı kullanmak bu yükü azaltıyor, insanın verimliliğini artırıyor. Ayrıca o dönemde hesaplama hızının pratik sınırı zaten donanım değil insandı.

Soru 6

EN: *What hardware capability that first appeared in the IBM 704 computer strongly affected the evolution of programming languages? Explain why.*

TR: IBM 704 bilgisayarında ilk kez ortaya çıkan hangi donanım özelliği programlama dillerinin evrimini güçlü biçimde etkiledi? Açıklayın.

● Cevap / Yönlendirme

IBM 704'te ortaya çıkan indeks yazmaçları (index registers) ve kayan noktalı donanım (floating-point hardware) dil evrimini derinden etkiledi. İndeks yazmaçları döngüleri ve dizi erişimini doğrudan donanımda destekledi; bu da derleyici tasarımcılarının verimli makine kodu üretmesini kolaylaştırdı. Fortran'ın donanım verimliliğini ön plana çıkaran tasarımı büyük ölçüde bu olanaklara dayanır.

Soru 7

EN: *In what year was the Fortran design project begun?*

TR: Fortran tasarım projesi hangi yılda başlatıldı?

● Cevap / Yönlendirme

Fortran tasarım projesi 1954 yılında IBM'de John Backus önderliğinde başlatıldı. İlk derleyici 1957'de tamamlandı ve kullanıma sunuldu. Bu, yüksek düzeyli dillerin pratikte kullanılabilirliğini kanıtlayan tarihi bir dönüm noktasıdır.

Soru 8

EN: *What was the primary application area of computers at the time Fortran was designed?*

TR: Fortran tasarlandığı dönemde bilgisayarların birincil uygulama alanı neydi?**● Cevap / Yönlendirme**

Fortran tasarlandığı dönemde (1950'lerin ortası) bilgisayarlar ağırlıklı olarak bilimsel ve mühendislik hesaplamaları için kullanılıyordu. Sayısal analiz, fizik simülasyonları, balistik hesaplamalar ve mühendislik problemleri bu alanın temel uygulamalarıydı. Fortran (Formula Translation) adı da bu bilimsel odaklanmayı yansıtır.

Soru 9

EN: What was the source of all of the control flow statements of Fortran I?

TR: Fortran I'in tüm kontrol akışı deyimlerinin kaynağı neydi?**● Cevap / Yönlendirme**

Fortran I'in tüm kontrol akışı deyimleri (koşullu dallanma, döngü vb.) doğrudan IBM 704'ün makine dili komutlarından türetilmiştir. Dil, donanımın yeteneklerini ve kısıtlamalarını birebir yansıtıyordu; bu nedenle üretilen makine kodu son derece verimli olabiliyordu.

Soru 10

EN: What was the most significant feature added to Fortran I to get Fortran II?

TR: Fortran I'den Fortran II'ye geçişte eklenen en önemli özellik neydi?**● Cevap / Yönlendirme**

Fortran II'ye geçişte eklenen en önemli özellik, bağımsız derlenen alt programlar (independently compiled subroutines) desteğidir. Bu özellik sayesinde büyük programlar parçalara bölünerek ayrı ayrı derlenebilir hâle geldi; bu da modüler programlamanın temelini attı.

Soru 11

EN: What control flow statements were added to Fortran IV to get Fortran 77?

TR: Fortran IV'ten Fortran 77'ye geçişte hangi kontrol akışı deyimleri eklendi?**● Cevap / Yönlendirme**

Fortran 77'ye geçişte eklenen önemli kontrol akışı öğeleri: (1) Tam anlamıyla yapılandırılmış IF-THEN-ELSE deyimini, (2) DO döngüsünün END DO ile kapatılabilmesi. Ayrıca karakter dizisi (character string) işleme yeteneği de bu sürümde önemli ölçüde geliştirildi.

Soru 12

EN: Which version of Fortran was the first to have any sort of dynamic variables?

TR: Dinamik değişkenlere ilk kez hangi Fortran sürümünde yer verildi?

● Cevap / Yönlendirme

Fortran 90, dinamik bellek tahsisini (dynamic memory allocation) destekleyen ALLOCATABLE dizileri ve işaretçileri (pointers) getiren ilk Fortran sürümüdür. Bu özellik, programların çalışma zamanında bellek ayırmasına olanak tanıdı.

Soru 13

EN: Which version of Fortran was the first to have character string handling?

TR: Karakter dizisi (character string) işlemeyi ilk kez hangi Fortran sürümünde ele alındı?

● Cevap / Yönlendirme

Fortran 77, kapsamlı karakter dizisi (character string) işleme desteği sunan ilk Fortran sürümüdür. Bu sürümde karakter türü tanımlandı ve dize karşılaştırma, atama gibi temel işlemler dile eklendi.

Soru 14

EN: Why were linguists interested in artificial intelligence in the late 1950s?

TR: 1950'lerin sonunda dilbilimciler neden yapay zekâya ilgi duydular?

● Cevap / Yönlendirme

1950'lerin sonunda dilbilimciler, doğal dil işleme (natural language processing) alanındaki çalışmalar nedeniyle yapay zekâya yöneldi. Özellikle otomatik çeviri (machine translation) projeleri büyük ilgi görüyordu. Dil yapılarını bilgisayarla analiz etmek ve anlamsal işlem yapabilmek için yeni hesaplama araçlarına ihtiyaç duyuluyordu; LISP bu boşluğu doldurmak üzere doğdu.

Soru 15

EN: Where was Lisp developed? By whom?

TR: Lisp nerede ve kim tarafından geliştirildi?

● Cevap / Yönlendirme

Lisp (List Processing), 1958 yılında MIT'de (Massachusetts Institute of Technology) John McCarthy tarafından geliştirildi. McCarthy, Lisp'i sembolik hesaplama ve yapay zekâ araştırmalarına uygun bir dil olarak tasarladı. Lisp, lambda hesabına (lambda calculus) dayanan fonksiyonel yaklaşımla programlama dili tarihine damgasını vurdu.

Soru 16

EN: In what way are Scheme and Common Lisp opposites of each other?

TR: Scheme ve Common Lisp birbirinin zıttı olmak bakımından nasıl farklılaşır?

● Cevap / Yönlendirme

Scheme son derece küçük ve minimalist bir dil olarak tasarlanmıştır; az sayıda temel yapı içerir ve akademik amaçlarla öğretim için uygundur. Common Lisp ise büyük, kapsamlı ve zengin özelliklere sahip endüstriyel bir dil olarak tasarlanmıştır. Scheme sadeliği ve temizliği, Common Lisp ise pratik kullanılabilirlik ve gücü ön plana çıkarır.

Soru 17

EN: *What dialect of Lisp is used for introductory programming courses at some universities?*

TR: Bazı üniversitelerde giriş programlama derslerinde Lisp'in hangi lehçesi kullanılmaktadır?

● Cevap / Yönlendirme

Scheme, Lisp'in sade ve temiz söz dizimiyle bazı üniversitelerde giriş programlama derslerinde tercih edilmektedir. MIT'nin efsanevi SICP (Structure and Interpretation of Computer Programs) dersi Scheme kullanır. Racket dili de Scheme'den türeyerek eğitim amaçlı yaygın biçimde kullanılmaktadır.

Soru 18

EN: *What two professional organizations together designed ALGOL 60?*

TR: ALGOL 60'ı birlikte hangi iki profesyonel kuruluş tasarladı?

● Cevap / Yönlendirme

ALGOL 60, ACM (Association for Computing Machinery — Amerika) ve GAMM (Gesellschaft für Angewandte Mathematik und Mechanik — Avrupa) adlı iki profesyonel kuruluşun ortak çalışmasıyla tasarlandı. Bu iş birliği, dilin uluslararası nitelik taşımasını sağlamış ve dil standardizasyonu açısından önemli bir emsal oluşturmuştur.

Soru 19

EN: *In what version of ALGOL did block structure appear?*

TR: Blok yapısı (block structure) ALGOL'un hangi sürümünde ortaya çıktı?

● Cevap / Yönlendirme

Blok yapısı (block structure), ALGOL 60'ta ortaya çıktı. Bu yapı sayesinde programcılar begin...end blokları içinde yerel değişkenler tanımlayabilir ve bu değişkenlerin kapsamını (scope) blokla sınırlayabilir hâle geldi. Blok yapısı, modern dillerdeki kapsam kurallarının temelini oluşturur.

Soru 20

EN: *What missing language element of ALGOL 60 damaged its chances for widespread use?*

TR: ALGOL 60'ta eksik olan hangi dil öğesi, dilin yaygınlaşma şansını zayıflattı?

● Cevap / Yönlendirme

ALGOL 60'ta standart bir giriş/çıkış (input/output — I/O) mekanizmasının bulunmaması, dilin yaygınlaşmasını büyük ölçüde engelledi. Her bilgisayar üreticisi kendi I/O uygulamasını geliştirmek zorunda kaldı; bu durum taşınabilirliği (portability) ciddi biçimde azalttı ve programcılar arasında tutarsız deneyimlere yol açtı.

Soru 21

EN: What language was designed to describe the syntax of ALGOL 60?

TR: ALGOL 60'ın sözdizimini (syntax) açıklamak için hangi dil tasarlandı?

● Cevap / Yönlendirme

BNF (Backus-Naur Form / Backus Normal Form), ALGOL 60'ın sözdizimini tanımlamak amacıyla John Backus ve Peter Naur tarafından geliştirildi. BNF, bağlamdan bağımsız gramerleri (context-free grammars) formal olarak ifade etmek için kullanılan ve o günden bu yana programlama dillerinin sözdizim tanımlamasında standart araç hâline gelen gösterim biçimidir.

Soru 22

EN: On what programming language was COBOL based?

TR: COBOL hangi programlama diline dayandırılmıştır?

● Cevap / Yönlendirme

COBOL, Grace Hopper tarafından geliştirilen FLOW-MATIC diline dayandırılmıştır. FLOW-MATIC, İngilizceye yakın sözdizimi (English-like syntax) ile iş uygulamalarına yönelik ilk üst düzey dillerden biriydi. COBOL bu yaklaşımı genişleterek iş ortamına uyarladı.

Soru 23

EN: In what year did the COBOL design process begin?

TR: COBOL tasarım süreci hangi yılda başladı?

● Cevap / Yönlendirme

COBOL tasarım süreci 1959 yılında başladı. ABD Savunma Bakanlığı'nın girişimiyle bir araya gelen CODASYL (Conference on Data Systems Languages) komitesi, ortak bir iş dili geliştirmek amacıyla çalışmalarını başlattı. İlk COBOL spesifikasyonu 1960'ta yayımlandı.

Soru 24

EN: What data structure that appeared in COBOL originated with Plankalkül?

TR: COBOL'da yer alan ve köken olarak Plankalkül'e dayanan veri yapısı hangisidir?

● Cevap / Yönlendirme

Kayıt (record) veri yapısı, Plankalkül'de tanımlanmış olup COBOL aracılığıyla geniş kitlelere ulaştı. COBOL'daki DATA DIVISION bölümündeki hiyerarşik kayıt yapısı bu mirası taşır. Plankalkül'ün kayıt yapısı, modern dillerdeki struct ve class yapılarının öncüsü sayılır.

Soru 25

EN: What organization was most responsible for the early success of COBOL (in terms of extent of use)?

TR: COBOL'un erken dönem başarısından (kullanım yaygınlığı bakımından) en çok hangi kuruluş sorumluydu?

● Cevap / Yönlendirme

ABD Savunma Bakanlığı (U.S. Department of Defense — DoD), COBOL'un yaygınlaşmasında belirleyici rol oynadı. DoD, 1960 yılında tüm askeri bilgisayarlarda COBOL kullanılmasını zorunlu kıldı ve bu dili desteklemeyen bilgisayarlara satın almayı reddetti. Bu politika, üreticileri ve kullanıcıları COBOL'u benimsemeye adeta mecbur bıraktı.

Soru 26

EN: What user group was the target of the first version of Basic?

TR: Basic'in ilk sürümünün hedef kullanıcı kitlesi kimlerdi?

● Cevap / Yönlendirme

Basic (Beginner's All-purpose Symbolic Instruction Code), 1964 yılında John Kemeny ve Thomas Kurtz tarafından Dartmouth College'da geliştirildi. Hedef kitle, bilgisayar bilimi ya da fen bilimleri öğrencisi olmayan, bilgisayarı bir araç olarak kullanmak isteyen genel eğitim öğrencileriydi (liberal arts students).

Soru 27

EN: Why was Basic an important language in the early 1980s?

TR: Basic, 1980'lerin başında neden önemli bir dil hâline geldi?

● Cevap / Yönlendirme

Basic, kişisel bilgisayar (PC) devrimiyle birlikte 1980'lerin başında kritik bir öneme kavuştu. Apple II, TRS-80 ve Commodore 64 gibi ilk kişisel bilgisayarlar ROM'larında Basic yorumlayıcısıyla birlikte geliyordu. Bu sayede kullanıcılar bilgisayarlarını açtıklarında doğrudan programlamaya başlayabiliyordu; Basic, ev kullanıcıları için programlamayı demokratikleştirdi.

Soru 28

EN: PL/I was designed to replace what two languages?

TR: PL/I hangi iki dilin yerine geçmek üzere tasarlandı?

● Cevap / Yönlendirme

PL/I, hem Fortran'ın (bilimsel hesaplama) hem de COBOL'un (iş uygulamaları) yerini almak amacıyla IBM tarafından tasarlandı. IBM, tek bir dilin her iki alanı da kapsayabileceğini düşünerek 1960'ların başında bu kapsamlı dili geliştirmeye başladı. Ancak bu iddialı hedef tam anlamıyla gerçekleşmedi.

Soru 29

EN: For what new line of computers was PL/I designed?

TR: PL/I hangi yeni bilgisayar serisi için tasarlandı?

● Cevap / Yönlendirme

PL/I, IBM'in 1964'te duyurduğu System/360 bilgisayar serisi için tasarlandı. System/360, hem bilimsel hem de ticari hesaplama gereksinimleri için tek bir donanım ailesi sunuyordu; PL/I de bu felsefeyi yazılım düzeyinde yansıtmak için geliştirildi.

Soru 30

EN: What features of SIMULA 67 are now important parts of some object-oriented languages?

TR: SIMULA 67'nin hangi özellikleri günümüzde nesne yönelimli dillerin önemli bileşenleri hâline gelmiştir?

● Cevap / Yönlendirme

SIMULA 67'nin nesne yönelimli programlamaya kazandırdığı temel özellikler: (1) Sınıflar (classes) ve nesneler (objects), (2) Kalıtım (inheritance), (3) Coroutine'ler (birlikte yürütme birimleri). Bu kavramlar Smalltalk, C++, Java ve Python gibi modern nesne yönelimli dillerin çekirdeğini oluşturur. SIMULA 67, nesne yönelimli paradigmanın atası kabul edilir.

Soru 31

EN: What innovation of data structuring was introduced in ALGOL 68 but is often credited to Pascal?

TR: ALGOL 68'de tanıtılan ancak genellikle Pascal'a atfedilen veri yapılandırma yeniliği nedir?

● Cevap / Yönlendirme

Kullanıcı tanımlı veri türleri (user-defined data types), ALGOL 68'de tanıtılmasına karşın bu yenilik çoğu zaman Pascal'a atfedilir. Bunun nedeni Pascal'ın daha geniş kitlelere ulaşmış ve bu özelliği popülerleştirmiş olmasıdır. Pascal'ın TYPE bildirimi, kullanıcıların kendi veri türlerini tanımlamasını olanaklı kıldı.

Soru 32

EN: What design criterion was used extensively in ALGOL 68?

TR: ALGOL 68'de kapsamlı biçimde kullanılan tasarım kriteri nedir?

● Cevap / Yönlendirme

Ortogonalite (orthogonality), ALGOL 68'in temel tasarım kriteridir. Bu ilke gereğince dildeki temel yapılar ve veri türleri herhangi bir kısıtlama olmaksızın her kombinasyonda bir araya getirilebilir. Ancak bu ilkenin aşırı uygulanması dilin öğrenilmesini ve kullanılmasını son derece zorlaştırdı.

Soru 33

EN: *What language introduced the case statement?*

TR: **case** deyimini ilk kez hangi dil tanıttı?

● Cevap / Yönlendirme

case deyimini (switch/case olarak da bilinir) programlama diline ilk olarak ALGOL W kazandırdı ve ardından Pascal tarafından popülerleştirildi. case deyimini, çok yönlü koşullu dallanmayı zincirleme if-else yapılarına kıyasla çok daha okunabilir biçimde ifade etmeye olanak tanır.

Soru 34

EN: *What operators in C were modeled on similar operators in ALGOL 68?*

TR: **C'deki hangi operatörler ALGOL 68'deki benzer operatörler model alınarak oluşturuldu?**

● Cevap / Yönlendirme

C'deki birleşik atama operatörleri (compound assignment operators) olan +=, -=, *=, /= ve benzerleri, ALGOL 68'deki operatörlerden esinlenilerek tasarlandı. Ayrıca ++ ve -- (artırma/azaltma) operatörleri de ALGOL 68'in etkisini taşır.

Soru 35

EN: *What are two characteristics of C that make it less safe than Pascal?*

TR: **C'yi Pascal'a kıyasla daha az güvenli kılan iki özellik nedir?**

● Cevap / Yönlendirme

(1) Tür sistemi zayıflığı: C'de işaretçi (pointer) aritmetiğine izin verilir; bu durum herhangi bir bellek konumuna erişmeyi mümkün kılar ve güvenlik açıklarına zemin hazırlar. (2) Dizi sınır denetimi yokluğu: C, dizi erişimlerinde otomatik sınır kontrolü yapmaz; bu durum tampon taşması (buffer overflow) gibi ciddi güvenlik sorunlarına yol açabilir.

Soru 36

EN: *What is a nonprocedural language?*

TR: **Prosedürel olmayan (nonprocedural) bir dil nedir?**

● Cevap / Yönlendirme

Prosedürel olmayan (nonprocedural / declarative) diller, programcının ne yapılmasını istediğini (what) belirttiği, nasıl yapılacağını (how) ise sistemin belirlediği dillerdir. SQL ve Prolog bu kategorinin örnekleridir. İmperatif dillerde ise programcı adım adım nasıl yapılacağını belirtmek zorundadır.

Soru 37

EN: What are the two kinds of statements that populate a Prolog database?

TR: Prolog veri tabanını dolduran iki tür deyim (statement) hangisidir?

● Cevap / Yönlendirme

Prolog veri tabanı iki tür ifadeden oluşur: (1) Olgular (facts) — doğru kabul edilen önermeleri tanımlar; örn. `anne(fatma, ali)`. (2) Kurallar (rules) — koşullu ilişkileri tanımlar; örn. `büyükanne(X,Z) :- anne(X,Y), anne(Y,Z)`. Bu iki yapıyla mantıksal ilişkiler modellenir ve sorgular bu ilişkiler üzerinden yanıtlanır.

Soru 38

EN: What is the primary application area for which Ada was designed?

TR: Ada hangi birincil uygulama alanı için tasarlandı?

● Cevap / Yönlendirme

Ada, gömülü gerçek zamanlı sistemler (embedded real-time systems) için tasarlandı; özellikle askeri sistemler ve kritik görev uygulamaları (safety-critical applications) ön planda tutuldu. ABD Savunma Bakanlığı (DoD), dağınık hâldeki yüzlerce programlama dilini tek bir standart dille değiştirmek amacıyla Ada'yı geliştirdi.

Soru 39

EN: What are the concurrent program units of Ada called?

TR: Ada'nın eşzamanlı (concurrent) program birimlerine ne ad verilir?

● Cevap / Yönlendirme

Ada'da eşzamanlı program birimleri görevler (tasks) olarak adlandırılır. Görevler (tasks), birbirleriyle bağımsız olarak yürütülebilen ve birbirleriyle randevu (rendezvous) mekanizması aracılığıyla iletişim kuran program birimleridir.

Soru 40

EN: What Ada construct provides support for abstract data types?

TR: Soyut veri türlerini (abstract data types) destekleyen Ada yapısı hangisidir?

● Cevap / Yönlendirme

Paketler (packages), Ada'da soyut veri türlerini destekleyen temel yapıdır. Bir paket, veri yapısını ve bu yapı üzerinde çalışan işlemleri bir arada kapsülleyerek (encapsulate) uygulama ayrıntılarını gizler ve yalnızca bir arayüz (interface) sunar. Bu yapı modern nesne yönelimli dillerdeki sınıf (class) kavramının öncüsüdür.

Soru 41

EN: *What populates the Smalltalk world?*

TR: **Smalltalk dünyasını ne oluşturur?**

● Cevap / Yönlendirme

Smalltalk dünyası yalnızca nesnelerden (objects) oluşur. Tam anlamıyla saf nesne yönelimli (pure object-oriented) bir dil olan Smalltalk'ta her şey — tam sayılar, sınıflar, metodlar, hatta kontrol yapıları bile — bir nesnedir. Bu bütünsel yaklaşım, nesne yönelimli paradigmanın en tutarlı biçimde uygulandığı örneklerden birini sunar.

Soru 42

EN: *What three concepts are the basis for object-oriented programming?*

TR: **Nesne yönelimli programlamanın temeli olan üç kavram nedir?**

● Cevap / Yönlendirme

Nesne yönelimli programlamanın (OOP) üç temel kavramı: (1) Kapsülleme (encapsulation) — veri ve işlemleri bir arada saklama ve uygulama ayrıntılarını gizleme; (2) Kalıtım (inheritance) — mevcut sınıftan yeni sınıf türetme ve özellik aktarma; (3) Çok biçimlilik (polymorphism) — aynı mesajın farklı nesnelerde farklı biçimlerde yanıtlanması.

Soru 43

EN: *Why does C++ include the features of C that are known to be unsafe?*

TR: **C++, C'nin güvensiz olduğu bilinen özelliklerini neden içermektedir?**

● Cevap / Yönlendirme

C++, C koduyla geriye dönük uyumluluğu (backward compatibility) korumak amacıyla C'nin güvensiz özelliklerini bünyesinde tutmuştur. Bu karar, mevcut C kodunun ve programcı kitlesinin C++'a sorunsuzca geçişini sağladı. Bjarne Stroustrup, performans ve uyumluluk hedeflerini güvenlik endişelerinin önüne koydu.

Soru 44

EN: *What language was Swift designed to replace?*

TR: **Swift hangi dilin yerini almak üzere tasarlandı?**

● Cevap / Yönlendirme

Swift, Apple ekosisteminde Objective-C'nin yerini almak üzere 2014 yılında Apple tarafından tasarlandı. Swift, Objective-C'ye kıyasla daha modern, güvenli ve okunabilir bir sözdizimi sunarken iOS, macOS ve diğer Apple platformlarındaki uygulama geliştirme deneyimini iyileştirmeyi hedefledi.

Soru 45

EN: What do the Ada and COBOL languages have in common?

TR: Ada ve COBOL dillerinin ortak özelliği nedir?

● Cevap / Yönlendirme

Ada ve COBOL'un temel ortak özelliği, her ikisinin de ABD Savunma Bakanlığı (U.S. Department of Defense — DoD) sponsorluğunda geliştirilmiş olmalarıdır. Her iki dil de DoD'un dağınık ve tutarsız yazılım geliştirme ortamını standartlaştırma çabasının ürünüdür. Bu ortak köken, her iki dilin de geniş kurumsal destek görmesini sağladı.

Soru 46

EN: What was the first application for Java?

TR: Java'nın ilk uygulama alanı neydi?

● Cevap / Yönlendirme

Java başlangıçta interaktif televizyon (interactive television / set-top boxes) ve gömülü tüketici elektroniği cihazları için tasarlandı (1991, Green Project). Ancak bu pazar beklenen ölçüde gelişmeyince Java, 1990'ların ortasında patlayan internet ve web uygulamaları için yeniden konumlandırıldı ve applet teknolojisiyle büyük ilgi gördü.

Soru 47

EN: What characteristic of Java is most evident in JavaScript?

TR: Java'nın hangi özelliği JavaScript'te en belirgin biçimde kendini gösterir?

● Cevap / Yönlendirme

Java'nın sözdizimsel özellikleri (C tabanlı küme parantez sözdizimi, blok yapısı) JavaScript'te en belirgin biçimde görülür. Ancak iki dilin isim benzerliğine karşın gerçekte birbirinden çok farklı tasarım felsefeleri vardır. JavaScript'in adı, Java'nın o dönemki popüleritesinden yararlanmak amacıyla pazarlama kararıyla belirlenmiştir.

Soru 48

EN: How does the typing system of PHP and JavaScript differ from that of Java?

TR: PHP ve JavaScript'in tiplendirme sistemi (typing system) Java'ninkinden nasıl farklılaşır?

● Cevap / Yönlendirme

PHP ve JavaScript dinamik tiplendirme (dynamic typing) kullanır; değişkenlerin türü derleme zamanında değil, çalışma zamanında belirlenir ve aynı değişken farklı türlerde değerler taşıyabilir. Java ise statik tiplendirme (static typing) kullanır; tüm değişken türleri derleme zamanında açıkça bildirilmek zorundadır. Dinamik tiplendirme esneklik sağlarken tür hatalarının tespitini güçleştirir.

Soru 49

EN: What array structure is included in C# but not in C, C++, or Java?

TR: C#'ta bulunan ancak C, C++ ya da Java'da yer almayan dizi yapısı hangisidir?

● Cevap / Yönlendirme

C#'ta çok boyutlu dikdörtgen diziler (rectangular multidimensional arrays) bulunur; C, C++ ve Java'da ise yalnızca düzensiz diziler (jagged arrays — dizi dizileri) desteklenir. C#'ın dikdörtgen dizilerinde tüm satırlar aynı uzunluktadır ve bellek düzeni daha öngörülebilirdir.

Soru 50

EN: What two languages was the original version of Perl meant to replace?

TR: Perl'ün ilk sürümü hangi iki dilin yerini almak amacıyla tasarlandı?

● Cevap / Yönlendirme

Perl, Larry Wall tarafından 1987 yılında metin işleme ve sistem yönetimi görevlerinde sed ve awk programlarının yerini almak için geliştirildi. Perl, bu iki aracın yeteneklerini tek bir güçlü dilde birleştirdi ve Unix betik (scripting) ortamında hızla yaygınlaştı.

Soru 51

EN: For what application area is JavaScript most widely used?

TR: JavaScript en yaygın olarak hangi uygulama alanında kullanılmaktadır?

● Cevap / Yönlendirme

JavaScript, istemci tarafı web programlaması (client-side web programming) alanında en yaygın kullanılan dildir. Web tarayıcılarında doğrudan çalışabilen tek programlama dili olma özelliğiyle web sayfalarına dinamizm, etkileşim ve kullanıcı arayüzü zenginliği kazandırır. Node.js ile sunucu tarafında da (server-side) yaygınlaşmıştır.

Soru 52

EN: What is the relationship between JavaScript and PHP, in terms of their use?

TR: JavaScript ve PHP, kullanım açısından birbirleriyle nasıl bir ilişki içindedir?

● Cevap / Yönlendirme

JavaScript ve PHP web geliştirme sürecinde birbirini tamamlayan (complementary) diller olarak işlev görür. JavaScript istemci tarafında (client-side / tarayıcıda) çalışırken, PHP sunucu tarafında (server-side) çalışır. Bir web uygulamasında PHP sunucuda veriyi işleyip HTML üretirken, JavaScript bu HTML sayfasını kullanıcının tarayıcısında etkileşimli kılar.

Soru 53

EN: PHP's primary data structure is a combination of what two data structures from other languages?

TR: PHP'nin birincil veri yapısı, diğer dillerden alınan hangi iki veri yapısının birleşimidir?

● Cevap / Yönlendirme

PHP'nin birincil veri yapısı olan ilişkilendirilebilir dizi (associative array), geleneksel dizi (array) ile karma tablosunun (hash table / dictionary) birleşimidir. PHP'de tek bir veri yapısı hem sayısal indeksli hem de anahtar-değer (key-value) çiftleri biçiminde kullanılabilir; bu esneklik PHP'yi web programlaması için son derece pratik kılar.

Soru 54

EN: What data structure does Python use in place of arrays?

TR: Python, dizilerin (arrays) yerine hangi veri yapısını kullanır?

● Cevap / Yönlendirme

Python, geleneksel dizilerin yerine listeler (lists) kullanır. Python listeleri heterojen olabilir (farklı türde öğeler içerebilir), dinamik olarak büyüüp küçülebilir ve dilimleme (slicing) gibi güçlü işlemleri destekler. Sayısal hesaplama için ise NumPy kütüphanesinin ndarray yapısı tercih edilir.

Soru 55

EN: What characteristic does Ruby share with Smalltalk?

TR: Ruby, Smalltalk ile hangi özelliği paylaşır?

● Cevap / Yönlendirme

Ruby, Smalltalk ile saf nesne yönelimlilik (pure object-oriented) özelliğini paylaşır. Her ikisinde de tam sayılar ve diğer ilkel (primitive) tipler dahil her şey bir nesnedir. Ruby'de `5.times { puts 'merhaba' }` gibi bir ifade yazılabilmesi, tam sayıların da nesne olduğunu gösterir.

Soru 56

EN: What characteristic of Ruby's arithmetic operators makes them unique among those of other languages?

TR: Ruby'nin aritmetik operatörlerinin hangi özelliği onu diğer dillerden farklı kılar?

● Cevap / Yönlendirme

Ruby'de aritmetik operatörler (+, -, *, / gibi) aslında metodlardır (methods). Bu operatörler yeniden tanımlanabilir (overridable) ve her biri bir nesne üzerinde çağrılan metod çağrısı olarak işlev görür. Bu yaklaşım, operatörlerin de nesne yönelimli sistemin tutarlı bir parçası olmasını sağlar.

Soru 57

EN: What deficiency of the switch statement of C is addressed with the changes made by C# to that statement?

TR: C'nin switch deyimindeki hangi eksiklik, C#'ın bu deyimde yaptığı değişikliklerle giderilmiştir?

● Cevap / Yönlendirme

C'nin switch deyiminde her case bloğunun sonuna break konulmazsa, yürütme bir sonraki case'e düşer (fall-through davranışı). Bu durum sık yapılan bir hatadır. C# bu sorunu, switch case'lerinin birinden diğerine otomatik geçişini (fall-through) varsayılan olarak yasaklayarak çözdü. Kasıtlı geçiş isteniyorsa açıkça goto case kullanılması zorunludur.

Soru 58

EN: What is the primary platform on which C# is used?

TR: C# ağırlıklı olarak hangi platform üzerinde kullanılmaktadır?

● Cevap / Yönlendirme

C#, Microsoft'un .NET platformu üzerinde birincil dil olarak kullanılmaktadır. Windows uygulamaları, ASP.NET web uygulamaları, Unity oyun motoru ve Xamarin mobil geliştirme başlıca kullanım alanlarıdır. .NET'in platform bağımsız sürümü olan .NET Core ile C# Linux ve macOS'ta da kullanılabilir. .NET'in platform bağımsız sürümü olan .NET Core ile C# Linux ve macOS'ta da kullanılabilir.

Soru 59

EN: What are the inputs to an XSLT processor?

TR: Bir XSLT işlemcisinin (processor) girdileri nelerdir?

● Cevap / Yönlendirme

Bir XSLT işlemcisi iki girdi alır: (1) Dönüştürülecek XML belgesi ve (2) Dönüşüm kurallarını tanımlayan XSLT stil sayfası (stylesheet). İşlemci bu iki girdiyi birleştirerek istenen çıktıyı üretir.

Soru 60

EN: What is the output of an XSLT processor?

TR: Bir XSLT işlemcisinin çıktısı nedir?

● Cevap / Yönlendirme

XSLT işlemcisinin çıktısı genellikle başka bir XML belgesi, HTML ya da düz metin (plain text) olabilir. En yaygın kullanımda XML verisini HTML'ye dönüştürmek için XSLT kullanılır; bu şekilde aynı veri farklı sunum biçimlerine kolayca uyarlanabilir.

Soru 61

EN: What element of the JSTL is related to a subprogram?

TR: JSTL'nin hangi ögesi alt programla (subprogram) ilişkilidir?

● Cevap / Yönlendirme

JSTL'deki (JavaServer Pages Standard Tag Library) fonksiyonlar (functions), diğer dillerdeki alt programlara (subprograms) karşılık gelir. Bu fonksiyonlar JSP sayfaları içinde yeniden kullanılabilir hesaplama birimleri olarak işlev görür.

Soru 62

EN: To what is a JSP document converted by a JSP processor?

TR: Bir JSP belgesi, JSP işlemcisi tarafından neye dönüştürülür?

● Cevap / Yönlendirme

JSP (JavaServer Pages) belgesi, JSP işlemcisi tarafından önce bir Java servlet'ine (Java sınıfına) dönüştürülür, ardından bu servlet derlenerek JVM üzerinde çalıştırılabilir bytecode hâline getirilir. Bu dönüşüm sürecinde JSP'deki HTML ve Java kodları birleştirilerek dinamik web içeriği üretilir.

Soru 63

EN: Where are servlets executed?

TR: Servlet'ler nerede yürütülür?

● Cevap / Yönlendirme

Servlet'ler, web sunucusunda (server-side) yürütülür. Özellikle Apache Tomcat veya GlassFish gibi Java EE uyumlu uygulama sunucularında (application servers) çalışırlar. İstemci (tarayıcı) bir istek gönderdiğinde servlet sunucuda çalışır ve ürettiği yanıtı (genellikle HTML) istemciye geri iletir.

BÖLÜM 2 — PROBLEM SETİ (Problem Set) • 24 Soru**Problem 1**

EN: What features of Plankalkül do you think would have had the greatest influence on Fortran 0 if the Fortran designers had been familiar with Plankalkül?

TR: Fortran tasarımcıları Plankalkül'ü bilseydi, Plankalkül'ün hangi özelliklerinin Fortran 0 üzerinde en büyük etkiyi yapacağını düşünüyorsunuz?

● Cevap / Yönlendirme

Plankalkül'ün dizi ve kayıt (record) veri yapıları Fortran'ın erken sürümlerini derinden etkileyebilirdi. Plankalkül'ün güçlü ifade yapısı ve matematiksel notasyon yakınlığı, Fortran'ın sözdizimini daha zengin kılabilirdi. Ayrıca Plankalkül'ün tür sistemi Fortran'ın ilk sürümünde eksik kalan güvenlik özelliklerini kazandırabilirdi.

Problem 2

EN: Determine the capabilities of Backus's 701 Speedcoding system, and compare them with those of a contemporary programmable hand calculator.

TR: Backus'un 701 Speedcoding sisteminin yeteneklerini belirleyin ve bunları günümüzdeki programlanabilir el hesap makineleriyle karşılaştırın.

● Cevap / Yönlendirme

701 Speedcoding sistemi, IBM 701 üzerinde kayan noktalı aritmetik ve sınırlı döngü yapıları sunuyordu; ancak belleği son derece kısıtlıydı ve yorumlama çok yavaştı. Günümüz programlanabilir hesap makineleri (TI-84 vb.) Speedcoding'e kıyasla çok daha hızlı, daha geniş belleğe sahip ve karmaşık programlamayı destekler niteliktedir. Bu karşılaştırma 60 yıllık teknoloji evriminin ne denli çarpıcı olduğunu gözler önüne serer.

Problem 3

EN: Write a short history of the A-0, A-1, and A-2 systems designed by Grace Hopper and her associates.

TR: Grace Hopper ve ekibi tarafından tasarlanan A-0, A-1 ve A-2 sistemlerinin kısa tarihini yazın.

● Cevap / Yönlendirme

A-0 (1951-1952), Grace Hopper tarafından geliştirilen ve sembolik matematiksel kodları makine koduna çeviren ilk derleyici benzeri sistemdir. A-1, A-0'ın iyileştirilmiş hâlidir. A-2 (1953) ise daha gelişmiş bir derleyicidir; programcılara alt program kütüphaneleri kullanma imkânı tanımış ve modern derleyicilerin kavramsal temelini atmıştır. Bu sistemler doğrudan FLOW-MATIC ve dolayısıyla COBOL'un gelişimine zemin hazırladı.

Problem 4

EN: Compare the facilities of Fortran 0 with those of the Laning and Zierler system.

TR: Fortran 0'ın olanaklarını Laning ve Zierler sistemiyle karşılaştırın.

● **Cevap / Yönlendirme**

Laning ve Zierler sistemi (1952, MIT), algebraik formülleri doğrudan makine koduna çeviren ilk sistemlerden biridir ve Fortran'ı önclemiştir. Fortran 0 ise IBM 704'ün donanım özelliklerine sıkı biçimde bağlıydı ve derlenmiş kodun verimliliğini ön plana çıkarıyordu. Laning-Zierler daha genel bir yaklaşım sunarken Fortran 0 performansa odaklanmıştır.

Problem 5

EN: Which of the three original goals of the ALGOL design committee, in your opinion, was most difficult to achieve at that time?

TR: ALGOL tasarım komitesinin üç özgün hedefinden hangisinin o dönemde gerçekleştirilmesinin en güç olduğunu düşünüyorsunuz?

● **Cevap / Yönlendirme**

ALGOL'un üç hedefi: (1) Makineden bağımsız (machine-independent) bir dil olması, (2) Algoritmik ifade için güçlü bir dil sağlaması, (3) İnsan okumasına uygun olması. Bu üç hedef arasında makine bağımsızlığını gerçek anlamda sağlamak en güçtü; çünkü standart bir I/O tanımlamak donanım farklılıkları nedeniyle imkânsıza yakındı ve bu eksiklik ALGOL'un yaygınlaşmasını engelledi.

Problem 6

EN: Make an educated guess as to the most common syntax error in Lisp programs.

TR: Lisp programlarındaki en yaygın sözdizim hatasına ilişkin bilgiye dayalı bir tahmin yapın.

● **Cevap / Yönlendirme**

Lisp'teki en yaygın sözdizim hatası, parantezlerin eşleşmemesidir (unbalanced parentheses). Lisp'in tüm sözdizimi iç içe parantezlere dayanır; fazladan ya da eksik bir parantez, programın tüm yapısını bozabilir. Bu yüzden Lisp'e bazen şakacı biçimde 'Lots of Irritating Superfluous Parentheses' (Çok Fazla Sinir Bozucu Gereksiz Parantez) denilir.

Problem 7

EN: Lisp began as a pure functional language but gradually acquired more and more imperative features. Why?

TR: Lisp saf fonksiyonel bir dil olarak başladı ancak zamanla giderek daha fazla imperatif özellik edindi. Neden?

● **Cevap / Yönlendirme**

Pratik programlama gereksinimleri Lisp'i fonksiyonel saflıktan uzaklaştırdı. Kullanıcılar; yan etkili işlemler, döngüler, değişken atama ve I/O gibi imperatif yapılara ihtiyaç duyuyordu. Saf fonksiyonel yaklaşım teorik açıdan zarif olsa da gerçek dünya uygulamalarında kullanım zorluğu yaratıyordu. Common Lisp bu iki paradigmayı birleştirerek çok-paradigmalı bir dil hâline geldi.

Problem 8

EN: Describe in detail the three most important reasons, in your opinion, why ALGOL 60 did not become a very widely used language.

TR: Sizce ALGOL 60'ın yaygın bir dil hâline gelememesinin en önemli üç nedenini ayrıntılı biçimde açıklayın.

● Cevap / Yönlendirme

(1) Standart I/O yokluğu: Taşınabilir giriş/çıkış mekanizması olmaması, farklı sistemlerdeki uygulamaların birbirleriyle uyumsuz olmasına yol açtı. (2) IBM'in desteğini vermemesi: O dönemin en baskın bilgisayar üreticisi IBM, kendi ürünleri için Fortran'ı desteklemeyi sürdürdü. (3) Avrupa merkezli kalması: ALGOL Avrupa akademisyenleri arasında güçlü destek gördü, ancak büyük ABD pazarına yeterince nüfuz edemedi.

Problem 9

EN: Why, in your opinion, did COBOL allow long identifiers when Fortran and ALGOL did not?

TR: Sizce COBOL uzun tanımlayıcılara (long identifiers) neden izin verirken Fortran ve ALGOL buna izin vermedi?

● Cevap / Yönlendirme

COBOL'un tasarım hedefi, iş dünyasındaki kişilerin kodu okuyup anlayabilmesiydi. Uzun ve açıklayıcı isimler (ör. GROSS-ANNUAL-SALARY), kodun kendi kendini belgelemesini (self-documenting code) sağlar. Fortran ve ALGOL ise bilimsel programcılara yönelikti; bu kişiler kısa matematiksel değişken adlarına alışkındı. Ayrıca o dönemde kısa tanımlayıcılar derleme süreçlerini de kolaylaştırıyordu.

Problem 10

EN: Outline the major motivation of IBM in developing PL/I.

TR: IBM'in PL/I geliştirmedeki temel motivasyonunu ana hatlarıyla özetleyin.

● Cevap / Yönlendirme

IBM'in PL/I'yı geliştirmedeki temel motivasyonu, müşteri tabanını tek bir güçlü dil etrafında toplamaktı. Bilimsel kullanıcılar Fortran'a, iş kullanıcıları ise COBOL'a bağlıydı; bu durum IBM'in iki ayrı derleyici ekosistemini sürdürmesini gerektiriyordu. PL/I ile IBM, her iki alanı kapsayan birleşik bir çözüm sunarak hem geliştirme maliyetlerini düşürmeyi hem de müşteri bağımlılığını artırmayı hedefledi.

Problem 11

EN: Was IBM's assumption, on which it based its decision to develop PL/I, correct, given the history of computers and language developments since 1964?

TR: IBM'in PL/I geliştirme kararının dayandığı varsayım, 1964'ten bu yana yaşanan bilgisayar ve dil gelişmeleri göz önüne alındığında doğru muydu?

● **Cevap / Yönlendirme**

IBM'in varsayımı — tek bir dilin tüm uygulama alanlarını karşılayabileceği — büyük ölçüde yanlış çıktı. Tarihsel süreç, farklı alanlara özel dillerin (C, Python, SQL, JavaScript, R vb.) gelişmeye devam ettiğini gösterdi. Bugün özelleşmiş dillerin bolluğu, tek bir evrensel dilin hayatta kalmasından çok dil çeşitliliğinin kazandığına işaret etmektedir.

Problem 12

EN: Describe, in your own words, the concept of orthogonality in programming language design.

TR: Programlama dili tasarımında ortogonallik (orthogonality) kavramını kendi cümlelerinizle açıklayın.

● **Cevap / Yönlendirme**

Ortogonallik, bir dildeki az sayıda temel yapının birbirleriyle tutarsızlık yaratmadan her türlü kombinasyonda kullanılabilmesi anlamına gelir. Tıpkı koordinat eksenlerinin birbirinden bağımsız olması gibi, ortogonal bir dilde her özellik diğerlerinden bağımsız biçimde çalışır. Ortogonal bir dil öğrenmesi daha kolay, davranışları daha öngörülebilir ve sürprizleri daha azdır.

Problem 13

EN: What is the primary reason why PL/I became more widely used than ALGOL 68?

TR: PL/I'nın ALGOL 68'den daha yaygın kullanılmasının birincil nedeni nedir?

● **Cevap / Yönlendirme**

PL/I'nın ALGOL 68'e kıyasla daha yaygın kullanılmasının birincil nedeni IBM'in güçlü kurumsal desteğidir. IBM, System/360 serisi için PL/I derleyicilerini bizzat geliştirdi ve müşterilerine aktif olarak pazarladı. ALGOL 68 ise akademik çevrelerde saygı görse de güçlü bir endüstriyel destekten yoksun kaldı ve son derece karmaşık yapısı nedeniyle yaygınlaşamadı.

Problem 14

EN: What are the arguments both for and against the idea of a typeless language?

TR: Tipsiz (typeless) bir dil fikrinin hem lehinde hem de aleyhinde ne gibi argümanlar vardır?

● **Cevap / Yönlendirme**

Lehinde: Daha az sözdizimsel ayrıntı gerektirir; hızlı prototiplemeye olanak tanır; değişkenler herhangi bir türde veri tutabildiğinden esneklik artar (örn. Python, JavaScript). Aleyhinde: Tür hataları derleme zamanında değil çalışma zamanında ortaya çıkar; bu da hata ayıklamayı güçleştirir. Büyük projelerde tür güvensizliği bakım maliyetini artırır ve bazı optimizasyonları imkânsız kılar.

Problem 15

EN: *Are there any logic programming languages other than Prolog?*

TR: **Prolog dışında mantık programlama dili var mıdır?**

● Cevap / Yönlendirme

Evet, Prolog dışında çeşitli mantık programlama dilleri mevcuttur: Datalog (veritabanı sorguları için), Mercury (güçlü tür sistemli, yüksek performanslı), Oz/Mozart, Curry (fonksiyonel-mantıksal hibrit) ve Answer Set Programming (ASP) sistemleri bu diller arasında sayılabilir. Ancak Prolog, mantık programlama paradigmasının tartışmasız en yaygın temsilcisi olmaya devam etmektedir.

Problem 16

EN: *What is your opinion of the argument that languages that are too complex are too dangerous to use, and we should therefore keep all languages small and simple?*

TR: **Çok karmaşık dillerin kullanımının tehlikeli olduğu ve bu nedenle tüm dilleri küçük ve sade tutmamız gerektiği argümanı hakkındaki görüşünüz nedir?**

● Cevap / Yönlendirme

Bu argüman güçlü bir çekirdeğe sahiptir: C ve C++'ın karmaşıklığından kaynaklanan güvenlik açıkları bu endişeyi haklı kılmaktadır. Öte yandan aşırı basitlik de kısıtlayıcı olabilir; karmaşık problemler ifade gücü yüksek diller gerektirebilir. İdeal yaklaşım, sade ama güçlü diller tasarlamaktır (Go, Rust gibi) — karmaşıklıkla değil, ortogonallik ve tutarlılıkla güç kazanan diller.

Problem 17

EN: *Do you think language design by committee is a good idea? Support your opinion.*

TR: **Komite tarafından dil tasarımının iyi bir fikir olduğunu düşünüyor musunuz? Görüşünüzü destekleyin.**

● Cevap / Yönlendirme

Komite tasarımının avantajları: Geniş paydaş katılımı, farklı kullanım alanlarının temsili ve uzlaşi yoluyla kabul görmesi. Dezavantajları: Tasarım tutarsızlıkları, uzlaşi adına gereksiz özellik çokluğu (ALGOL 68, PL/I gibi) ve karar alma süreçlerinin uzaması. Tek bir vizyoner tasarımcının eseri olan diller (C — Ritchie, Python — van Rossum) genellikle daha tutarlı bir felsefe sergiler.

Problem 18

EN: Languages continually evolve. What sort of restrictions do you think are appropriate for changes in programming languages?

TR: Diller sürekli evrilmektedir. Programlama dillerindeki değişiklikler için hangi tür kısıtlamaların uygun olduğunu düşünüyorsunuz?

● **Cevap / Yönlendirme**

Dil değişikliklerinde geriye dönük uyumluluk (backward compatibility) kritik bir kısıtlama olmalıdır; mevcut kod bozulmamalıdır. Yeni özellikler mevcut paradigmalara tutarlı ve öğrenme eğrisini gereğinden fazla artırmayan nitelikte olmalıdır. Fortran'ın evrimi bu konuda iyi bir örnek sunar: Her sürüm önceki kodu çalıştırmayı sürdürmüş, yeni özellikler kademeli biçimde eklenmiştir.

Problem 19

EN: Build a table identifying all of the major language developments, together with when they occurred, in what language they first appeared, and the identities of the developers.

TR: Tüm önemli dil gelişmelerini, ne zaman gerçekleştiklerini, hangi dilde ilk göründüklerini ve geliştiricilerinin kimliklerini içeren bir tablo oluşturun.

● **Cevap / Yönlendirme**

Bu soru kapsamlı bir araştırma tablosu gerektirmektedir. Temel başlıklar: Plankalkül (1945, Zuse), Fortran (1957, Backus/IBM), LISP (1958, McCarthy/MIT), COBOL (1959, CODASYL), ALGOL 60 (1960, ACM+GAMM), SIMULA 67 (1967, Dahl&Nygaard;), C (1972, Ritchie), Smalltalk (1972, Kay), Ada (1983, DoD), C++ (1985, Stroustrup), Python (1991, van Rossum), Java (1995, Gosling/Sun).

Problem 20

EN: There have been some public interchanges between Microsoft and Sun concerning the design of Microsoft's J++ and C# and Sun's Java. Read some of these documents and write an analysis.

TR: Microsoft ve Sun arasında J++, C# ve Java tasarımı konusunda kamuoyuna yansımış tartışmalar yaşandı. Bu belgeleri okuyun ve anlaşmazlıkları analiz edin.

● **Cevap / Yönlendirme**

Microsoft, Java'yı .NET platformuna entegre etmek için J++ geliştirdi ve Java'ya Windows'a özgü uzantılar ekledi; bu durum Sun'ın sözleşme ihlali iddiasıyla dava açmasına yol açtı. Sun, Java'nın platform bağımsızlığının bozulduğunu savundu. Bu anlaşmazlık Microsoft'un C# ve .NET'i geliştirme kararını hızlandırdı. Olay, açık standartlar ile ticari çıkarlar arasındaki gerilimin simgesi hâline geldi.

Problem 21

EN: In recent years data structures have evolved within scripting languages to replace traditional arrays. Explain the chronological sequence of these developments.

TR: Son yıllarda betik dillerinde veri yapıları geleneksel dizilerin yerini almak üzere evrildi. Bu gelişmelerin kronolojik sırasını açıklayın.

● Cevap / Yönlendirme

Betik dillerinde veri yapısı evrimi: Perl (1987) ilişkilendirilebilir dizileri (associative arrays / hash) yaygınlaştırdı. Python (1991) listeleri ve sözlükleri (dictionaries) temel yapılar olarak sundu. JavaScript (1995) nesne (object) tabanlı anahtar-değer yapısını öne çıkardı. PHP (1994) tüm dizi türlerini tek bir yapıda birleştirdi. Bu süreç geleneksel sabit tipli dizilerden dinamik, esnek veri yapılarına doğru bir geçişi yansıtır.

Problem 22

EN: Explain two reasons why pure interpretation is an acceptable implementation method for several recent scripting languages.

TR: Saf yorumlamanın (pure interpretation) pek çok modern betik dili için kabul edilebilir bir uygulama yöntemi olmasının iki nedenini açıklayın.

● Cevap / Yönlendirme

(1) Betik dilleri genellikle I/O yoğun görevler için kullanılır (dosya işleme, web istekleri); bu görevlerde hesaplama süresi değil I/O bekleme süresi baskındır, dolayısıyla yorumlama yavaşlığı belirgin biçimde hissedilmez. (2) Betik dillerinin güçlü noktası hızlı geliştirme ve prototiplemedir; derleme adımının olmaması geliştirme-test döngüsünü kısaltır ve programcı verimliliğini artırır.

Problem 23

EN: Why, in your opinion, do new scripting languages appear more frequently than new compiled languages?

TR: Sizce neden yeni betik dilleri, yeni derlenmiş dillerden daha sık ortaya çıkmaktadır?

● Cevap / Yönlendirme

Betik dili geliştirmek çok daha az çaba gerektirir: Derleyici yazmak karmaşık optimizasyon, bellek yönetimi ve platform uyumluluğu gerektirirken bir yorumlayıcı çok daha kısa sürede geliştirilebilir. Ayrıca betik dilleri belirli niş alanlara (web, sistem yönetimi, veri analizi) odaklanarak hızla benimsenir. Yüksek düzeyli çalışma zamanı ortamları (Python, Ruby VM) üzerine yeni diller kolayca inşa edilebilir.

Problem 24

EN: Give a brief general description of a markup-programming hybrid language.

TR: Bir biçimlendirme-programlama (markup-programming) karma dilini kısaca genel hatlarıyla tanımlayın.

● Cevap / Yönlendirme

Biçimlendirme-programlama karma dilleri, statik belge biçimlendirme ile dinamik programlama mantığını tek bir çatı altında birleştirir. JSP (JavaServer Pages), PHP ve ASP.NET bu kategorinin örnekleridir. HTML gibi biçimlendirme yapılarının içine yerleştirilen programlama kodu, sunucu tarafında dinamik içerik üretmek için kullanılır. XSLT ise XML dönüşümü için özelleşmiş bir karma dil olarak kabul edilebilir.

BÖLÜM 2 — PROGRAMLAMA ALIŞTIRMALARI (Programming Exercises) • 3 Alıştırma

Alıştırma 1

EN: To understand the value of records in a programming language, write a small program in a C-based language that uses an array of structs that store student information, including name, age, GPA as a float, and grade level as a string. Also write the same program without using structs.

TR: Programlama dillerinde kayıtların (records/structs) değerini anlamak için, öğrenci bilgilerini (isim, yaş, GPA ve sınıf düzeyi) depolayan struct dizisi kullanan bir program yazın; ardından aynı programı struct kullanmadan yazın.

● Cevap / Yönlendirme

Bu alıştırma, struct kullanımının kodun okunabilirliğini ve bakım kolaylığını ne ölçüde artırdığını deneyimsel biçimde gösterir. Struct'sız sürümde dört ayrı dizi (isim[], yas[], gpa[], sinif[]) kullanmak gerekir ve her öğrenci için indeksleri elle senkronize etmek zorunda kalırsınız. Struct kullanan sürüm ise veriyi mantıksal birimler hâlinde gruplandırır, kodu anlaşılır ve hata yapmaya daha az açık kılar.

Alıştırma 2

EN: To understand the value of recursion in a programming language, write a program that implements quicksort, first using recursion and then without recursion.

TR: Programlama dillerinde özyinelemenin (recursion) değerini anlamak için, quicksort algoritmasını önce özyineleme kullanarak ardından özyineleme kullanmadan uygulayan bir program yazın.

● Cevap / Yönlendirme

Bu alıştırma, özyinelemenin (recursion) kodun ifade gücünü nasıl artırdığını somut biçimde ortaya koyar. Özyinelemeli quicksort, algoritmanın mantığını doğrudan yansıtır: 'Bir öğeyi böl, sol parçayı sırala, sağ parçayı sırala.' Özyinelemesiz sürümde ise açık bir yığın (stack) veri yapısı tutmak, yığın yönetimini elle yapmak ve çok daha fazla kod yazmak gerekir; bu durum kodun anlaşılmasını güçleştirir.

Alıştırma 3

EN: To understand the value of counting loops, write a program that implements matrix multiplication using counting loop constructs. Then write the same program using only logical loops—for example, while loops.

TR: Sayma döngülerinin (counting loops) değerini anlamak için, matris çarpımını önce sayma döngüsü yapıları kullanarak ardından yalnızca mantıksal döngülerle (while döngüleri) uygulayan bir program yazın.

● Cevap / Yönlendirme

Bu alıştırma, for döngüsü gibi sayma döngülerinin döngü başlangıcı, koşulu ve artışını tek bir yerde ifade ederek okunabilirliği nasıl artırdığını gösterir. while döngüsü kullanan sürümde sayacı (i, j, k) ayrıca başlatmak, koşulu kontrol etmek ve artırmak gerekir; bu durum döngü mantığını gizler ve hata yapma olasılığını artırır. Matris çarpımı gibi iç içe indeks gerektiren algoritmalarda bu fark özellikle belirginleşir.